

Trabalho Prático 2

Ítalo Dell'Areti

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte – MG – Brasil

1. Introdução

O problema proposto consiste em desenvolver um sistema de comunicação cliente-servidor para simular o monitoramento e despacho de equipes de drones na região da Amazônia Legal. O sistema deve implementar um protocolo de rede customizado sobre UDP que permita a troca de mensagens entre estação de monitoramento (cliente) e central de comando (servidor), gerenciando missões de combate ao desmatamento. O protocolo deve resolver quatro aspectos principais:

- Estabelecer comunicação entre cliente e servidor usando UDP/IP com mecanismo de ACKs para confiabilidade.
- Implementar suporte dual-stack (IPv4 e IPv6) para máxima compatibilidade.
- Gerenciar despacho inteligente de drones usando algoritmo de Dijkstra para encontrar a equipe mais próxima disponível.
- Garantir sincronização correta através de múltiplas threads no cliente com mecanismos de mutex e condition variables.

Ao final, o sistema deve processar todas as telemetrias recebidas, despachar equipes para alertas detectados, simular missões com tempo aleatório e gerenciar a disponibilidade de recursos (equipes de drones). Portanto, com base nas características de protocolos de rede, arquitetura cliente-servidor, sockets UDP, multi-threading, foi implementado uma solução que respeita todas as nuances do problema.

2. Modelagem

2.1 Arquitetura do Sistema

O sistema segue o modelo cliente-servidor com comunicação UDP/IP, suportando IPv4 e IPv6. A estrutura modular é composta por cinco componentes principais:

- **common.h**: Definições compartilhadas incluindo constantes do protocolo, estruturas de mensagens (headers e payloads), enums para tipos de mensagens, ACKs e status de cidades, e estruturas auxiliares para representação do grafo.

- **rede.c/.h**: Camada de rede responsável por abstração da criação de sockets IPv4 e IPv6 tanto para servidor quanto para cliente, encapsulando as diferenças entre `AF_INET` e `AF_INET6`.
- **grafo.c/.h**: Implementação da estrutura de dados do grafo representando a Amazônia Legal, incluindo algoritmo de Dijkstra para cálculo de caminhos mínimos e função de busca da capital disponível mais próxima.
- **server.c**: Servidor single-threaded que aguarda mensagens UDP, processa telemetrias, detecta alertas, despacha drones e gerencia o estado global das equipes.
- **client.c**: Cliente multi-threaded (4 threads) que simula monitoramento de cidades, envia telemetrias periódicas, recebe ordens de drone e simula execução de missões.

A separação de responsabilidades garante que cada módulo tenha função específica: `common.h` define o protocolo, `rede.c` abstrai comunicação de rede, `grafo.c` implementa lógica geográfica e algoritmos, `server.c` gerencia estado e despacho, e `client.c` simula estação de monitoramento com comportamento concorrente.

2.2 Protocolo de Comunicação

O protocolo utiliza mensagens binárias estruturadas transmitidas via UDP, com sistema de ACKs para garantir confiabilidade. Cada mensagem é composta por um cabeçalho seguido de um payload específico ao tipo de mensagem.

Tipos de Mensagens Definidos:

1. **MSG_TELEMETRIA (1)**: Enviada pelo cliente periodicamente contendo status de todas as cidades monitoradas.
2. **MSG_ACK (2)**: Mensagem de confirmação bidirecional (cliente ↔ servidor).
3. **MSG_EQUIPE_DRONE (3)**: Ordem de despacho enviada pelo servidor ao cliente.
4. **MSG_CONCLUSAO (4)**: Notificação de conclusão de missão enviada pelo cliente ao servidor.

Payloads Definidos:

- **payload_telemetria_t**: Contém array de telemetrias (`id_cidade`, `status`) para todas as cidades monitoradas.
- **payload_ack_t**: Contém campo `status` indicando tipo do ACK (telemetria, equipe_drone ou conclusão).
- **payload_equipe_drone_t**: Contém `id_cidade`` (alerta) e `id_equipe` (capital despachada)
- **payload_conclusao_t**: Contém `id_cidade` e `id_equipe` da missão concluída.

Fluxo de Comunicação:

1. Cliente inicia e envia telemetria a cada 30 segundos
2. Servidor processa telemetria e responde com ACK_TELEMETRIA
3. Para cada alerta detectado, servidor envia MSG_EQUIPE_DRONE
4. Cliente recebe ordem, responde com ACK_EQUIPE_DRONE e inicia missão
5. Thread de simulação executa missão por tempo aleatório (1-30s)
6. Cliente envia MSG_CONCLUSAO ao término
7. Servidor libera equipe e responde com ACK_CONCLUSAO

2.3 Regras de Despacho

O sistema implementa as seguintes regras para gerenciamento de drones:

Recursos Disponíveis:

- 9 equipes de drones (uma em cada capital da Amazônia Legal)
- Cada equipe pode estar livre (disponível) ou ocupada (em missão)
- Array global equipes_livres[MAX_CIDADES] rastreia disponibilidade

Algoritmo de Despacho:

1. Quando alerta é detectado na cidade X:
 - Executa Dijkstra a partir da cidade X
 - Calcula distâncias para todas as capitais
 - Seleciona capital livre mais próxima (menor distância)
 - Marca equipe como ocupada
 - Envia ordem de drone ao cliente
2. Quando missão é concluída:
 - Servidor recebe MSG_CONCLUSAO
 - Libera equipe correspondente
 - Equipe volta a estar disponível para novos despachos

Casos Especiais:

- **Todas equipes ocupadas:** Servidor exibe "Nenhuma equipe disponível no momento" e não despacha
- **Capital em alerta:** Capital pode despachar equipe para si mesma (distância = 0 km)
- **Múltiplos alertas simultâneos:** Processados sequencialmente, cada um recebe equipe diferente se disponível

2.4 Comportamento Dual-Stack IPv4/IPv6

O sistema implementa suporte nativo para IPv4 e IPv6, com comportamentos distintos dependendo da versão do protocolo escolhida.

Servidor IPv4 (./server v4):

- Cria socket exclusivamente IPv4 usando AF_INET
- Bind em INADDR_ANY (0.0.0.0)
- Aceita apenas datagramas UDP de clientes IPv4

Servidor IPv6 (./server v6):

- Cria socket IPv6 usando AF_INET6
- Bind em in6addr_any
- Por padrão, habilita modo dual-stack (aceita IPv4 e IPv6)
- Não utiliza flag IPV6_V6ONLY, mantendo comportamento padrão do SO

Cliente IPv4 (./client v4):

- Conecta ao servidor via 127.0.0.1:8080
- Utiliza estrutura sockaddr_in
- Datagramas enviados com endereçamento IPv4

Cliente IPv6 (./client v6):

- Conecta ao servidor via ::1:8080
- Utiliza estrutura sockaddr_in6
- Datagramas enviados com endereçamento IPv6

Modo Dual-Stack:

Quando o servidor IPv6 está ativo, ele pode receber conexões tanto de clientes IPv6 (endereço nativos ::1) quanto de clientes IPv4 (mapeados como ::ffff:127.0.0.1). Esta decisão maximiza a compatibilidade sem adicionar complexidade ao código.

3. Implementação

3.1 Principais Decisões

A implementação do sistema de monitoramento de drones baseou-se em escolhas fundamentais que definiram a arquitetura e qualidade da solução.

Protocolo UDP com ACKs:

A decisão mais importante foi utilizar UDP como protocolo de transporte, implementando confiabilidade através de sistema de ACKs e retries.

Esta escolha foi motivada pela natureza dos dados (telemetria pode tolerar alguma perda) mas necessidade de garantir recebimento de ordens críticas (despacho de drones).

O sistema implementa retry com até 3 tentativas e timeout de 5 segundos, encontrando equilíbrio entre confiabilidade e simplicidade.

Multi-threading no Cliente:

Optou-se por arquitetura multi-threaded no cliente com 4 threads especializadas:

- **Thread 1 (Monitoramento):** Atualiza status aleatório das cidades a cada 30s
- **Thread 2 (Telemetria):** Envia telemetrias periódicas com sistema de retry
- **Thread 3 (Recepção):** Multiplexador que recebe TODAS as mensagens do servidor usando `select()` com timeout
- **Thread 4 (Simulação):** Executa missões de drones com tempo aleatório

Esta separação eliminou problemas de concorrência ao delegar responsabilidades claras. A Thread 3 funciona como dispatcher central, notificando Thread 2 sobre ACKs recebidos (via condition variable `cond_ack`) e processando ordens de drone que bloqueiam aguardando conclusão da Thread 4.

Sincronização Inter-Thread:

Implementou-se comunicação entre threads usando primitivas POSIX

3.2 Desafios e Soluções Implementadas

Sincronização entre Threads 3 e 4:

O principal desafio foi evitar deadlock quando Thread 3 aguarda conclusão de missão. A solução utilizou `pthread_cond_wait()` que libera atomicamente o mutex, permitindo que Thread 4 sinalize conclusão. Thread 4 executa `sleep()` de simulação sem segurar mutex, readquirindo-o apenas para notificar término via `cond_conclusao`.

Thread 3 como Multiplexador:

Inicialmente, threads separadas recebiam ACKs e ordens de drone, causando race condition no mesmo socket. Thread 3 foi redesenhada como multiplexador central usando `select()` com timeout de 1s, recebendo todas as mensagens e despachando para handlers apropriados (`processar_ack_recebido()` ou `processar_ordem_drone()`).

Sistema de Retry Seletivo:

Para evitar envios duplicados, isolou-se retry na função `enviar_com_retry()` chamada apenas por Thread 2 (telemetria). Mensagens de conclusão fazem envio único com timeout, sem loop de retry, sendo tolerantes a falhas de ACK.

Shutdown:

Signal handler para `SIGINT` seta flag `programa_ativo = 0`, faz broadcast de todas condition variables e chama `shutdown()` no socket. Thread 3 usa `select()` com timeout de 1s e sleeps foram substituídos por `sleep_interruptivel()` que verifica flag a cada segundo, permitindo encerramento responsivo.

Thread Ocupada:

Quando Thread 4 está em missão, Thread 3 verifica `missao_atual.ativa` antes de aceitar nova ordem. Se ocupada, envia `ACK_EQUIPE_DRONE` mas ignora ordem, evitando sobrescrever missão em andamento conforme especificação.

3.3 Estratégias de Organização

A estrutura foi organizada separando responsabilidades em camadas horizontais e verticais:

Camada de Protocolo (`common.h`):

- Definições compartilhadas entre cliente e servidor
- Independente de implementação de rede ou lógica de negócio
- Facilita evolução do protocolo sem quebrar compatibilidade

Camada de Rede (`rede.c/.h`):

- Abstração de diferenças entre IPv4/IPv6
- Funções simétricas: `criar_socket_servidor_ipv4/ipv6` e `criar_socket_cliente_ipv4/ipv6`
- Permite trocar implementação de socket sem afetar camadas superiores

Camada de Lógica (`grafo.c/.h`):

- Isolamento de algoritmos geográficos (Dijkstra, busca de capital)
- Reutilizável para outros contextos (não acoplado a drones)
- Funções puras que operam sobre estrutura de dados imutável

Camadas de Aplicação (`server.c`, `client.c`):

- Orquestram uso das camadas inferiores
- Implementam lógica específica de negócio (despacho, simulação)
- Servidor: arquitetura event-loop simples
- Cliente: arquitetura multi-threaded com clara separação de responsabilidades por thread

Nomenclatura Padronizada:

- Estruturas terminam em `_t` (ex: `header_t`, `payload_telemetria_t`)
- Funções descrevem ação + objeto (ex: `enviar_ack`, `processar telemetria`, `buscar_capital_livre`)
- Constantes em UPPER_CASE (ex: `MAX_CIDADES`, `TIMEOUT_ACK_MS`)
- Variáveis descritivas (ex: `equipes_livres`, `cliente_addr`, `missao_atual`)

Esta organização permitiu desenvolvimento incremental, testando cada camada isoladamente antes de integrá-la. A fase final foi dedicada exclusivamente a refatoração, eliminando código morto ou duplicado, garantindo a consistência do projeto.

4. Considerações Finais

O desenvolvimento deste trabalho proporcionou experiência prática valiosa na implementação de protocolos de rede sobre UDP, sincronização multi-thread com POSIX threads e design de sistemas distribuídos, especialmente na resolução de problemas relacionados à confiabilidade e coordenação de threads concorrentes.

A parte mais direta de implementar foi a estrutura básica de mensagens e o protocolo de comunicação UDP, por ser aplicação direta dos conceitos de sockets UDP combinado com structs em C. A implementação do suporte dual-stack IPv4/IPv6 também foi relativamente simples devido a implementações anteriores.

A principal dificuldade encontrada foi na sincronização correta entre as 4 threads do cliente, especialmente evitando deadlocks e race conditions. A Thread 3 precisava receber mensagens assincronamente enquanto Thread 2 aguardava ACKs específicos e Thread 4 executava missões. Esta parte demandou várias iterações até encontrar modelo de comunicação usando condition variables que satisfizesse todos os requisitos sem deadlocks.

A implementação do algoritmo de Dijkstra integrado com verificação de disponibilidade de equipes também apresentou desafios interessantes, principalmente em garantir que capital selecionada estivesse realmente livre e que distância calculada fosse mínima entre todas as opções válidas.

O sistema de retry com timeout para mensagens UDP exigiu atenção especial para evitar envios duplicados de ordens de drone ou conclusões de missão. A escolha por aplicar retry apenas em telemetrias (Thread 2) e envio único para outras mensagens resolveu este problema de forma limpa.

Por segurança, também foi verificada ausência de warnings de compilação usando flags `-Wall -Wextra -std=c99 -pthread` e validação manual do sistema através de diversas execuções e validação de telemetrias enviadas/recebidas, alertas detectados, drones despachados, missões concluídas e comportamento quando todas as equipes estão ocupadas para garantir funcionamento adequado.

5. Referências

[1] GEEKSFORGEEKS. **UDP Server-Client Implementation in C**. Disponível em: <https://www.geeksforgeeks.org/udp-server-client-implementation-c/>

[2] GEEKSFORGEEKS. **Socket Programming in C/C++**. Disponível em: <https://www.geeksforgeeks.org/socket-programming-cc/>

[3] LINUX FOUNDATION. **pthread_cond_wait(3) - Linux Manual Page**. Disponível em: https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html

[4] LINUX FOUNDATION. **pthread_mutex_lock(3) - Linux Manual Page**. Disponível em: https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html

[5] GEEKSFORGEEKS. **Thread Management Functions in C** Disponível em: <https://www.geeksforgeeks.org/c/thread-functions-in-c-c/>

[6] CORMEN, Thomas H. et al. **Algoritmos: Teoria e Prática**. 3ª ed. Elsevier, 2012. (Capítulo 24: Caminhos Mínimos de Única Origem - Dijkstra)